

Enhancing Graph-based Approximate Maximum Inner Product Search via Norm-Adaptive Partitioning

Abstract

The rapid development of vector databases and large language models has significantly increased the importance of vector search problems, particularly *Approximate Maximum Inner Product Search* (AMIPS) and *Approximate Nearest Neighbor Search* (ANNS). Over the past decade, various approaches have been proposed to efficiently address the ANNS and AMIPS problems. Compared to other methods, such as those based on Locality Sensitive Hashing (LSH), proximity graph-based methods have shown superior query performance on AMIPS. However, the performance of graph-based methods for AMIPS is still not fully optimized due to the norm bias issue, while graph-based methods have demonstrated strong effectiveness for ANNS.

In this paper, we theoretically analyze the norm bias in MIPS and identify a *norm domination* phenomenon: results are consistently dominated by a handful of large-norm vectors, with negligible contributions from small-norm ones. Building on this observation, we propose the Norm-Adaptive Partitioning (NAP) scheme, which splits the vectors in a dataset into HEAD, BODY, and TAIL based on vector norms. The HEAD contains few large-norm vectors for exhaustive search; the TAIL includes small-norm ones that can be safely pruned under accuracy bounds; and the BODY, characterized by concentrated norms, is well-suited for conventional proximity graphs. We further design a practical NAP algorithm that minimizes search cost while ensuring bounded accuracy. To demonstrate NAP, we introduce a hybrid index combining a lightweight structure (e.g., a hash table) for the HEAD and a proximity graph for the BODY. Experimental results demonstrate that NAP reduces the index size of existing proximity graphs by over 50%, and that the NAP-based hybrid index achieves more than 2x speedup over state-of-the-art graph-based AMIPS methods to reach the same recall target.

CCS Concepts

• **Do Not Use This Code** → **Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Maximum Inner Product Search, Graph based Method

ACM Reference Format:

. 2018. Enhancing Graph-based Approximate Maximum Inner Product Search via Norm-Adaptive Partitioning. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 13 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Vector search has emerged as a foundational capability underpinning a wide spectrum of modern applications, including recommendation systems [3, 20], web search [22, 37], natural language processing [12, 23], and multimodal retrieval [24, 39]. In such contexts, entities such as documents, products, and images are represented as high-dimensional vectors, and the system must efficiently retrieve the most relevant or similar vectors to a given query vector. As a result, the *Maximum Inner Product Search* (MIPS) and *Nearest Neighbor Search* (NNS) problems play a crucial role in these applications by finding a vector from a dataset $\mathcal{D}$ that maximizes the inner product with a query vector q or minimizes the Euclidean distance to q , respectively. With growing dataset sizes and dimensionality, exact vector search becomes computationally prohibitive due to the "curse of dimensionality" [2, 5, 16, 18]. Thus, approximate vector searches—*Approximate Nearest Neighbor Search* (ANNS) and *Approximate Maximum Inner Product Search* (AMIPS)—have become a practical alternative, offering dramatic gains in query efficiency while maintaining desirable retrieval quality.

A variety of approaches have been proposed to efficiently solve the ANNS and AMIPS problems over the past decade. These approaches can be broadly categorized into locality-sensitive hashing (LSH)-based methods [14, 16], graph-based methods [13, 27], and quantization-based methods [15, 19]. Compared to LSH-based and quantization-based methods, graph-based approaches have demonstrated superior query performance on the ANNS problem, owing to their strong ability to capture neighborhood structures within the dataset. To address the ANNS problem, graph-based methods construct an approximate proximity graph (APG), where each data point serves as a vertex, and edges connect vertices whose corresponding points are sufficiently close in Euclidean space. When a query q arrives, a greedy search [13, 27, 41] is performed starting from an entry point in the APG. The search terminates at a local minimum—a vertex whose distance to q is smaller than that of any of its neighbors in the graph—which is returned as the ANNS result. This strategy is grounded in the intuition that in Euclidean space, the nearest neighbors of a point's nearest neighbors are more likely to be nearest neighbors of the original point [11]. By leveraging this property, graph-based methods can retrieve high-quality ANNS results while probing only a small fraction of the dataset. Graph-based methods have also been successfully applied to the AMIPS problem. In this setting, an APG is built by connecting pairs of points with large inner products [28, 33], enabling a greedy search to identify candidates with high inner products.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

However, our empirical analysis on several datasets (e.g., Tiny5M) shows that graph-based methods perform considerably worse on MIPS. This demonstrates the crucial difference between the MIPS and ANNS problems. A notable observation is that as more points are explored, APGs steadily improve their accuracy on ANNS and eventually reach a performance plateau close to perfect accuracy. In contrast, their accuracy on MIPS remains consistently lower and fails to converge to a comparable level. This evident performance gap underscores the intrinsic difference between MIPS and ANNS, and highlights the inherent difficulty of effectively addressing the MIPS problem using APGs. Therefore, it is essential to develop an effective strategy that allows graph-based methods to achieve AMIPS performance comparable to their success on ANNS, thereby fully leveraging the advantages of proximity-graph-based search.

This naturally raises a critical question: *can we pinpoint and overcome the fundamental reason for the inadequate performance of graph-based methods in AMIPS?* We answer this affirmatively by uncovering a key insight—the norm domination phenomenon, where search results are overwhelmingly monopolized by a few large-norm vectors, while small-norm vectors contribute negligibly. This phenomenon fundamentally hinders proximity graphs, which are highly effective for ANNS, from achieving comparable performance on AMIPS. To understand this limitation, consider a dataset whose vectors have nearly uniform norms. In this case, maximizing the inner product is equivalent to minimizing the Euclidean distance, and graph-based methods such as APG can naturally perform AMIPS as effectively as ANNS. However, real-world datasets typically exhibit a wide range of norm distributions, leading to strong norm bias: points with extremely large or small norms distort the neighborhood structure, degrading graph connectivity and search accuracy. To tackle this bottleneck, we propose the Norm-Adaptive Partitioning (NAP) framework, which manages data points with varied norms and enables efficient and accurate AMIPS search. NAP strategically partitions the dataset into three norm-based segments—HEAD, BODY, and TAIL—in descending order of norms. The HEAD contains a small number of large-norm vectors that are highly likely to appear in MIPS results. Since they are few but critical, we search them exhaustively using lightweight methods such as linear scan or LSH, rather than indexing them in a proximity graph. The TAIL consists of small-norm vectors that almost never appear in MIPS results and can be safely pruned to save computation and space. The BODY, which forms the majority, has moderately varied norms and thus serves as the ideal candidate set for proximity graph construction. Building an APG on the BODY notably alleviates the adverse effect of norm domination and preserves high search efficiency. We further design practical NAP algorithms that determine the partition boundaries to minimize search cost under strict accuracy guarantees and generate high-quality entry points for the BODY graph search based on the HEAD results. Finally, we demonstrate the effectiveness of NAP through a hybrid index that integrates a lightweight hash table for the HEAD and a proximity graph for the BODY. Experimental results demonstrate that NAP reduces the index size of existing proximity graphs by over 50%, and that the NAP-based hybrid index achieves more than 2x speedup over state-of-the-art graph-based AMIPS methods to reach the same recall. In summary, this paper makes the following contributions.

- We conduct a critical analysis of the limitations inherent in graph-based methods for Maximum Inner Product Search (MIPS), identifying the norm domination phenomenon and clarifying its detrimental impact on both the distribution of MIPS results and the query performance of proximity graphs.
- We introduce a novel Norm-Adaptive Partitioning (NAP) strategy, which segments the dataset into three distinct regions—HEAD, BODY, and TAIL—based on vector norms, effectively countering the adverse effects of norm bias. Accompanying this strategy, we develop a practical partitioning algorithm that maximizes the identification of TAIL points while rigorously bounding accuracy loss within a predefined error threshold.
- We design hybrid index structures customized to the characteristics of the HEAD and BODY partitions, leveraging their respective sizes and norm distributions to enable efficient and accurate retrieval of the most relevant MIPS candidates.
- We conduct extensive experiments on real-world datasets to validate the effectiveness of NAP. The results demonstrate that NAP reduces the query time and memory overhead of state-of-the-art graph-based methods by more than 50% when the dataset exhibits a wide norm distribution.

The rest of the paper is organized as follows. Section 2 reviews related work on MIPS problem. Section 3 presents the problem definition and background. Section 4 provides an in-depth analysis of datasets and quantifies the norm domination phenomenon. Section 5 details the NAP strategy. Section 6 describes NAP-based index structure and the corresponding AMIPS query processing algorithm. Section 7 provides theoretical analysis and proofs of key lemmas. Section 8 reports extensive experimental evaluations, and Section 9 concludes the paper.

2 Related Work

As a subfield of vector similarity search, the AMIPS problem has attracted extensive attention and has been widely explored through LSH-based methods [4, 17, 21, 30, 31, 40, 42] and graph-based methods [7, 25, 28, 32–34].

However, LSH functions are not directly applicable in inner product spaces, as a point does not necessarily have the highest inner product with itself, despite always exhibiting the highest collision probability with itself. To address this inherent limitation, transformation-based techniques have been proposed to convert the AMIPS problem into ANNS problem, enabling it to be tackled via LSH-based methods [14, 16]. Such transformations typically involve two functions, $P(x)$ and $Q(q)$, which map data points and query vectors from the original inner product space $(\mathbb{R}^d, ip)$ into a new metric space $(\mathbb{R}^m, dist)$. Under these mappings, maximizing $q^\top x$ becomes equivalent to minimizing $dist(Q(q), P(x))$, allowing the AMIPS problem to be answered by performing ANNS on $Q(q)$ in the transformed space [4, 30, 31]. Nevertheless, these transformations often neglect norm bias, resulting in distance distortions [17]. That is, points with distinct inner products relative to the query may end up at similar distances from the query in the new space, rendering the transformed ANNS ineffective. To mitigate this issue, several works [17, 40, 42] propose partitioning the dataset into blocks based on vector norms, ensuring that points within each block share similar magnitudes. Consequently, ANNS queries within a single block

are less affected by distortion. However, this design necessitates searching across multiple blocks, which increases query costs compared to querying the entire dataset at once, thereby creating a trade-off between accuracy and efficiency.

Given the proven effectiveness of graph-based methods such as HNSW [27] and DiskANN [32] in ANNS, applying them to AMIPS is a promising direction. A straightforward strategy involves constructing APGs directly in the inner product space. This can be achieved by modifying existing frameworks (e.g., replacing the metric function in HNSW with the inner product, known as ip-NSW [28]), a practice commonly adopted in industrial systems due to its simplicity. However, since the inner product is non-metric and the MIPS neighbor of a point x 's MIPS neighbor is not x 's the MIPS neighbor again, greedy search in APGs for AMIPS suffers from degraded performance, primarily due to norm bias. To address this, several studies [33, 34, 43] propose improved edge selection strategies that prioritize large-norm points. While beneficial in some cases, these methods often fail when the dataset exhibits a wide norm distribution. In such cases, large-norm points disproportionately attract incoming edges, creating “traps” that hinder search efficiency. Additionally, the PSP approach [7] suggests that a sufficiently large scalar μ can be chosen such that for any query q , the ANNS result of $q' = \mu \cdot q$ over $\mathcal{D}$ coincides with the AMIPS result for q . This enables the AMIPS problem to be simplified into an ANNS query for q' . However, this scaling operation inflates the norm of q' beyond the typical range of dataset vectors, causing the distances between q' and dataset points to become nearly uniform. As a result, this method suffers from the same type of distance distortion seen in transformation-based approaches. In fact, q' becomes an out-of-distribution point, and such queries have been demonstrated to challenge the performance of ANNS algorithms [6].

3 Preliminaries

In this paper, we focus on the AMIPS problem in memory. Let $\mathbb{R}^d$ be a d -dimensional vector space, $x_1^\top x_2$ represent the inner product between points $x_1, x_2 \in \mathbb{R}^d$, and $\langle x_1, x_2 \rangle$ denote the angle between x_1 and x_2 . We define the AMIPS problem as follows:

Definition 3.1 ((c, k) -Approximate Maximum Inner Product Search). Given a dataset $\mathcal{D}$, a query point $q \in \mathbb{R}^d$, an approximation ratio $0 < c < 1$, and a positive integer k , a (c, k) -approximate maximum inner product search (AMIPS) returns k points $x_1, \dots, x_k \in \mathcal{D}$, sorted in descending order according to their inner products with the query point q . For each $i = 1, \dots, k$, the following condition holds: $q^\top x_i \geq c \cdot q^\top x_i^*$, where $q^\top x_i^*$ represents the i -th maximum inner product in the dataset.

The ratio c denotes the approximation property of AMIPS and is not directly utilized in graph-based methods. Next, we describe some properties of the normal distribution, which are used to prove the correctness of our framework.

Definition 3.2 (i.i.d. d -dimensional normal random vector). A d -dimensional point $q = (q_1, \dots, q_d)$ is an i.i.d. d -dimensional normal random vector if each of its entries q_i is independently normally distributed as $N(\mu, \sigma^2)$, where μ and σ^2 are the expectation and variance of q_i , respectively. We denote this as $q \sim N^d(\mu, \sigma^2)$.

LEMMA 3.3 ([8, 9]). *The sum $S = \sum_{i=1}^d a_i X_i$ follows a normal distribution $N(0, \sum_{i=1}^d a_i^2)$, where $X_i \sim N(0, 1)$ and $a_i \in \mathbb{R}$.*

LEMMA 3.4 (INDEPENDENCE [8, 10]). *If X and Y are jointly normal and uncorrelated, then they are independent.*

4 Motivation

Norm bias is a commonly discussed phenomenon in the MIPS problem [17, 40, 42], which indicates that points with larger norms are more likely to exhibit a larger inner product with a query than points with smaller norms. Consequently, it is more promising to identify the MIPS results from points with larger norms. However, few studies quantitatively analyze the probability that MIPS results originate from points with large norms, which makes it difficult to fully leverage the effect of norm bias for MIPS queries. In this section, we aim to model the norm bias quantitatively, beginning with a simple case of two points and extending the analysis to two sets of points with disjoint norm ranges. Next, we investigate the norm distribution and MIPS result distribution in real datasets to uncover the dominance of points with large norms in MIPS results. Finally, we explore the impact of norm bias on the graph structure to demonstrate the inefficiency of MIPS queries in APGs.

4.1 Norm Domination

To formalize norm bias, we begin by considering a simplified scenario with two points, x_1 and x_2 , where $r_1 = \|x_1\|$ and $r_2 = \|x_2\|$, with $r_1 > r_2$. Based on the norm bias phenomenon, for any query q , it is reasonable to infer that x_1 is more likely to yield a larger inner product with q than x_2 .

LEMMA 4.1. *Consider two orthogonal random points $x_1, x_2 \in \mathbb{R}^d$ with $r_1 = \|x_1\|$, $r_2 = \|x_2\|$, and an i.i.d. d -dimensional normal random point $q \sim N^d(0, 1)$. If $q^\top x_1 > 0$ and $q^\top x_2 > 0$, then the probability that x_1 has a larger inner product with q than x_2 is $\frac{2}{\pi} \cdot \arctan\left(\frac{r_1}{r_2}\right)$.*

The proof of Lemma 4.1 is presented in Section 7.1. Lemma 4.1 quantifies the norm bias phenomenon by revealing the probability that x_2 becomes the MIPS result of q when another point x_1 exists. To simplify the model, we assume that the maximum inner product is positive. Based on this lemma, it is evident that points like x_1 , with a larger norm, are more likely to be the MIPS result, and the likelihood increases with the ratio of their norms, r_1/r_2 . Thus, when the norms of the two points differ significantly, we can compute how small the probability of x_2 becoming the MIPS result can be.

Estimation of the domination probability. However, the dataset $\mathcal{D}$ typically contains multiple points, and it is essential to analyze the norm bias in this case. To simplify the analysis, we divide $\mathcal{D}$ into two parts and compute the probability that the MIPS result falls into one of them. Let us consider two sets of random non-zero points $X = \{x_1, x_2, \dots, x_{n_1}\}$ and $Y = \{y_1, y_2, \dots, y_{n_2}\}$ with $r_1 = \min_{x \in X} \|x\|$ and $r_2 = \max_{y \in Y} \|y\|$. For an i.i.d. d -dimensional normal random query point $q \sim N^d(0, 1)$, let x^* be the MIPS neighbor of q in the set $X \cup Y$, we aim to estimate the probability that $x^* \in X$ when $\gamma = r_1/r_2 > 1$. Based on Lemma 3.3, we have $I_i = q^\top x_i \sim N(0, \|x_i\|^2)$. To simply estimate the cumulative density function (cdf) of $I_X^* = \max_{x \in X} q^\top x$, we ignore the correlations between any

two variables I_i and I_j , then for any a number t ,

$$\begin{aligned} \Pr[I_X^* \leq t] &= \Pr[\max I_i \leq t] \\ &= \Pr[I_1 \leq t, I_2 \leq t, \dots, I_i \leq t] \\ &= \prod_{i=1}^{n_1} \Pr[I_i \leq t] = \prod_{i=1}^{n_1} \Phi\left(\frac{t}{\|x_i\|}\right) \end{aligned} \quad (1)$$

where $\Phi(t)$ is the cumulative distribution function (cdf) of the standard normal distribution. So, the cdf of I_X^* is $F_{I_X^*}(t) = \prod_{i=1}^{n_1} \Phi\left(\frac{t}{\|x_i\|}\right)$. Similarly, for $I_Y^* = \max_{y \in Y} q^\top y$, its cdf is $F_{I_Y^*}(t) = \prod_{i=1}^{n_2} \Phi\left(\frac{t}{\|y_i\|}\right)$. Assume I_X^* and I_Y^* are independent, we have

$$\begin{aligned} \Psi(X, Y) &= \Pr[I_X^* > I_Y^*] \\ &= \int_{-\infty}^{+\infty} \int_{t_1}^{+\infty} F'_{I_X^*}(t_2) dt_2 \cdot F'_{I_Y^*}(t_1) dt_1. \end{aligned} \quad (2)$$

$\Psi(X, Y)$ forms an estimator of the probability that $x^* \in X$.

LEMMA 4.2. Denote $\gamma = r_1/r_2$ where $r_1 = \min_{x \in X} \|x\|$ and $r_2 = \max_{y \in Y} \|y\|$, $\Psi(X, Y)$ is at least

$$F(\gamma, n_1, n_2) = \int_0^{+\infty} \left[1 - \Phi\left(\frac{t}{\gamma}\right) \right] d\Phi^{n_2}(t). \quad (3)$$

The proof of Lemma 4.2 can be found in Section 7.1. Lemma 4.2 provides strong evidence for the norm bias phenomenon between two sets of points, X and Y , when the norm of points in X is at least γ times larger than that in Y . It computes the lower bound of the probability that the MIPS result of a random query belongs to X rather than Y as $F(\gamma, n_1, n_2)$. $F(\gamma, n_1, n_2)$ converges rapidly to 1 with increasing n_1 or γ when $\gamma > 1$. Even if n_1 is only a few thousand, $F(\gamma, n_1, n_2)$ can approach 1 when $\gamma = 2.0$ and $n_2 = 1$ million. We refer to this phenomenon as *norm domination*.

Definition 4.3 (Norm Domination). Consider two random point sets, $X = \{x_1, x_2, \dots, x_{n_1}\}$ and $Y = \{y_1, y_2, \dots, y_{n_2}\}$, where $r_1 = \min_{x \in X} \|x\|$ and $r_2 = \max_{y \in Y} \|y\|$. When $\gamma = r_1/r_2$ is sufficiently large, the MIPS result of any query q in $X \cup Y$ will almost always appear in X , even if $|X| \ll |Y|$. We refer to this phenomenon as *norm domination*, where Y is dominated by X .

In contrast to norm bias, norm domination offers a more precise characterization: rather than simply stating that points with larger norms are more likely to appear among MIPS results, it asserts that once the norm gap exceeds a certain threshold, e.g., γ , the probability that the MIPS result falls within the large-norm subset approaches nearly 100%, even when that subset accounts for only about 1% of the points. This conclusion is striking, as it implies that with the presence of large-norm points, MIPS results can be found by searching only thousands of points, rather than the entire dataset of several million points, with high probability. In the following section, we present the distributions of MIPS results in real datasets to further illustrate norm domination.

4.2 Norm Domination in Real Datasets

We conduct an empirical analysis on five real datasets – Audio, MNIST, YahooMusic, Gist1M, and Tiny5M – to investigate whether these datasets exhibit skewed norm distributions and the relationship between MIPS results and the norm of points. Specifically,

we first characterize the overall norm distributions to identify potential imbalances, and then quantify the proportion of k -MIPS results contributed by large-norm vectors. As shown in Figure 1, three key observations emerge: (1) all datasets display non-uniform, right-skewed norm distributions; (2) a small fraction of large-norm vectors dominate the top- k MIPS results; and (3) the degree of norm imbalance varies significantly across datasets. Detailed analyses of these findings are presented below.

Norm skewness across all datasets. The top row of Figure 1 shows the empirical histograms of vector norms for each dataset, with the median indicated by the *dashed red line*. First, all datasets exhibit highly non-uniform and asymmetric norm distributions, rather than flat or symmetric ones, aligning with our assumption on the norm of points. Even in relatively concentrated datasets such as Audio and MNIST, the ratios between the 99-th and 10-th percentiles ($2.06/0.83 \approx 2.5\times$ and $3378/1785 \approx 1.9\times$, respectively) reveal persistent asymmetry. Second, the majority of vectors cluster around relatively small magnitudes, while a small fraction extend into a long right tail with substantially larger norms. For example, although the norms in Audio span from 0.3 to 3.1, over 75% of them fall below 1.45, and only the top 1% reach 2.06; a similar situation is even more pronounced in the YahooMusic dataset: norms in YahooMusic span from 0.0017–0.34, but 75% of them fall below 0.0178, and only the top 1% reach 0.08, indicating the highly concentrated and long-tail distribution of norms in real datasets. This disparity in real datasets demonstrates that, while the norm of most points lies in compact small regions, a small subset of large-norm vectors dominate the upper tail, creating the structural basis for norm domination in the MIPS problem.

Concentration of MIPS results on large-norm points. To quantify how the observed norm skewness affects MIPS retrieval, we examine the cumulative contribution of large-norm vectors to the top- k MIPS results of all query points. As shown in the bottom row of Figure 1, each curve depicts the fraction of retrieved top- k neighbors that originate from the top- $x\%$ of vectors ranked by their norms. A steeper curve indicates a stronger dominance of large-norm points in retrieval outcomes, reflecting how severely the query results are biased toward the large-norm region. The top-1 and top-10 curves rise sharply within the first few percent of the points, indicating that only a tiny portion of large-norm vectors contribute to retrieval outcomes. For instance, in Tiny5M and Gist, more than 99% of the top-10 MIPS neighbors are drawn from the top 0.05% of large-norm points, while even the broader top-100 set is almost entirely covered by the top 0.1%. Audio and MNIST exhibit a milder but still significant bias, where 70–80% of the top-10 results originate from the top 10% of large-norm points. YahooMusic lies between these extremes, showing strong skewness for top-10 retrieval (over 99% within the top 5%) but a smoother distribution for larger k .

Dataset-dependent skewness intensity. Although norm domination is universally observed, its severity varies markedly across datasets. YahooMusic, Gist, and Tiny5M exhibit highly skewed norm distributions, where over 90% of top-10 MIPS results arise from the top 1–2% of the largest-norm points. In contrast, Audio and MNIST display more moderate imbalance—approximately 70–80% of retrieved results are concentrated within the top 10% of

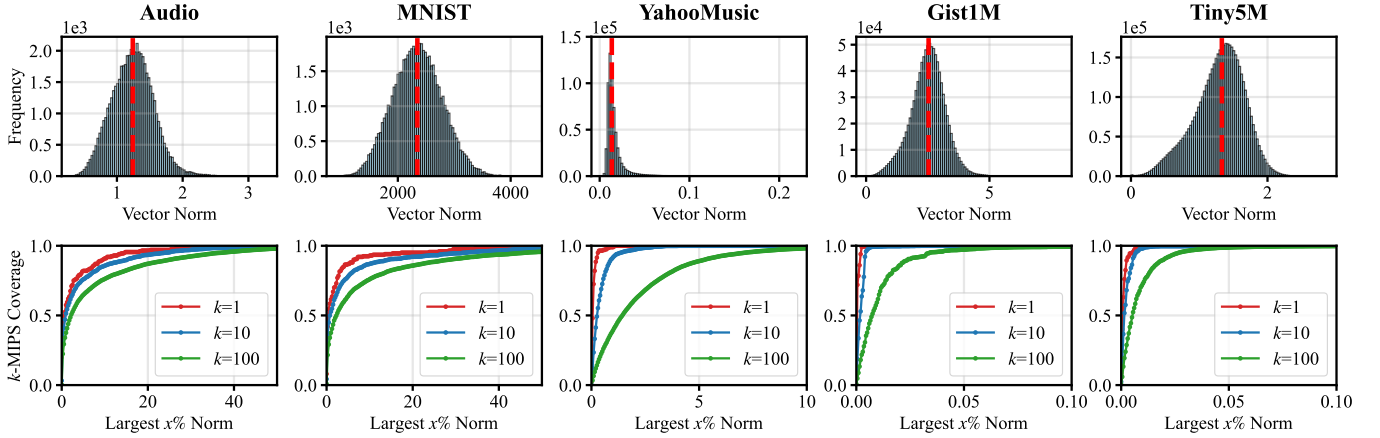


Figure 1: Norm Distribution and MIPS Distribution in Real Datasets

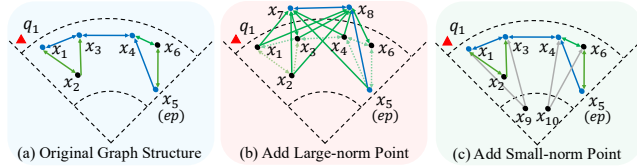


Figure 2: Illustration of how vector norms influence graph structure (max out-degree is set to 2).

vectors—showing that smaller-norm regions still contribute meaningfully to retrieval. This dataset-dependent disparity suggests that the number of dominating vectors varies with the underlying norm distribution, motivating the need for a norm-adaptive strategy that facilitates the efficient AMIPS query based on the specific norm distribution in the datasets.

Example 4.4. These experimental findings also validate the theoretical norm-domination model introduced in Lemma 4.2. Specifically, we verify this correspondence by splitting each dataset into two subsets according to norms: the top 10% large-norm vectors as X and the bottom 50% as Y , from which we estimate the norm ratio γ . In the highly skewed dataset YahooMusic, where $n_1 = 62,476$ with minimum norm $r_1 = 0.026$ and $n_2 = 312,380$ with maximum norm $r_2 = 0.013$, we obtain $\gamma = r_1/r_2 = 2.0$, the theoretical domination probability is $F(\gamma, n_1, n_2) \approx 0.999$. In contrast, for the relatively balanced dataset Audio ($n_1 = 5,418$, $r_1 = 1.63$; $n_2 = 27,093$, $r_2 = 1.24$), $\gamma = 1.31$ yields $F(\gamma, n_1, n_2) \approx 0.946$. This aligns closely with the empirical observation that more than 99% of top-10 MIPS neighbors originate from the top 1% of large-norm vectors in the YahooMusic dataset, while the proportion decreases to 70–80% in the Audio dataset. These results demonstrate that the observed empirical dominance ratios quantitatively conform to the theoretical prediction, reinforcing the validity of the proposed norm-domination model in practical MIPS scenarios.

4.3 The Effect of Norm Domination on APG

The theoretical analysis of norm domination, coupled with the empirical evidence of severe retrieval concentration in real datasets, demonstrates that MIPS results are overwhelmingly concentrated on a small subset of large-norm points. This phenomenon of norm

domination not only biases retrieval results but also fundamentally alters the structure of the APG for MIPS. To illustrate this, we compare the changes in graph structures and query performance in APGs when large-norm or small-norm points are inserted, as shown in Figure 2.

Syphon effect of large-norm points in APGs. According to Lemma 4.1, large-norm points tend to produce high inner products with most other points. As a result, these points are frequently selected as neighbors and accumulate an exceptionally high number of in-edges, ultimately becoming *hub nodes*. As illustrated in Figure 2(b), the insertion of large-norm nodes x_7, x_8 traps almost all the in-edges in the APG. Many modest and small-norm nodes (e.g., x_1, x_2, x_3, x_4, x_6) replace their original local edges (such as $x_3 \leftrightarrow x_4$ or $x_1 \leftrightarrow x_2$) with edges directed toward x_7 or x_8 in order to maximize the inner product. Hub nodes x_7 and x_8 attract a large number of in-edges from x_1-x_6 , resulting in a sharp spike in in-degrees at the head. In contrast, the in-degrees of x_1-x_6 decrease because their original in-edges are evicted due to the out-degree constraint of their neighbors, thus compromising the connectivity in the APG. These two phenomena collectively undermine the quality of the APG for MIPS queries. As a result, the greedy search for AMIPS is quickly “pulled” toward the hub region $\{x_7, x_8\}$ and struggles to escape due to the lack of connectivity, leading to low query accuracy for certain query points.

Futility of small-norm points in APGs. In contrast, small-norm vectors produce low inner products with other nodes, and thus receive few in-edges. As depicted in Figure 2(c), when small-norm points x_9, x_{10} are inserted, most of their out-edges point to higher-norm nodes (e.g., x_1, x_3, x_4 , and x_6), with no edges directed back to them. As a result, x_9 and x_{10} remain at the periphery of the graph, forming dead-end branches that are never visited during the search unless they are entry points. These small-norm nodes contribute little to the overall connectivity or navigability of the graph but still incur storage and construction overhead without improving retrieval accuracy.

Example 4.5. Figure 2 illustrates the AMIPS query under three scenarios: (a) a balanced APG, (b) the APG after inserting large-norm points, and (c) the APG after inserting small-norm points. Each node has an out-degree of 2. In (a), all nodes $\{x_1, \dots, x_6\}$

have comparable norms, forming a well-connected structure. A greedy search beginning from x_5 follows the path $x_5 \rightarrow x_4 \rightarrow x_3 \rightarrow x_1$, successfully reaching the correct result. In (b), adding large-norm nodes x_7, x_8 causes all existing nodes to redistribute their edges toward the hub nodes x_7, x_8 . The hubs, in turn, primarily connect to each other or to a few nearby nodes ($x_7 \rightarrow x_8, x_8 \rightarrow x_3$). The search path thus becomes $x_5 \rightarrow x_8 \rightarrow x_7$, unable to reach x_1 , demonstrating how hub dominance traps the search in the head region. In (c), the insertion of small-norm nodes x_9, x_{10} results in isolated low-degree nodes without altering connectivity. Due to their small inner products with other points, no other nodes select them as neighbors, leaving them with zero in-degree. The search path remains unchanged. These nodes consume storage while contributing little to retrieval accuracy.

5 The Norm-Adaptive Partitioning Framework

As analyzed in Section 4, the issue of norm domination significantly impacts the MIPS problem. First, a substantial number of small-norm points are rarely returned as MIPS results for any query. Therefore, identifying and excluding these points from the index can enhance query efficiency. Second, large-norm points tend to attract a disproportionately high number of edges, resulting in an imbalanced edge distribution within the APGs. To address these challenges and improve MIPS query efficiency, we propose a norm-adaptive partitioning strategy that determines which data points should be stored in the APGs.

5.1 Norm-Adaptive Partitioning Scheme

Given a dataset $\mathcal{D} = \{x_1, x_2, \dots, x_n\}$ sorted in non-increasing order of norms ($\|x_i\| \geq \|x_{i+1}\|$), the norm-adaptive partition (NAP) divides $\mathcal{D}$ into three segments—HEAD, BODY, and TAIL—such that:

- The HEAD consists of large-norm points $\{x_1, \dots, x_{n_1}\}$. As shown in Figure 1, due to the long-tail distribution, certain points exhibit significantly larger norms than the rest. These points are highly likely to appear as MIPS results, making it reasonable to prioritize searching within this segment. Early identification of candidates from HEAD can either terminate the query promptly or guide subsequent exploration in other regions.
- The BODY includes the medium-norm points from x_{n_1+1} to x_{n-n_2} . This segment forms the core of the dataset and typically exhibits a narrower norm range than HEAD and TAIL (see Figure 1). Although their norms are smaller than those in HEAD, these points can still become MIPS results for some queries q , particularly if their cosine similarity with q is high. To capture such cases, we construct an APG over BODY. By excluding large- and small-norm points, the resulting APG suffers less from norm domination, leading to improved search effectiveness.
- The TAIL consists of small-norm points ranging from x_{n-n_2+1} to x_n . Its size n_2 is determined by the size of HEAD (n_1) and the norm ratio $\gamma = \frac{\|x_{n_1}\|}{\|x_{n-n_2+1}\|}$. Based on Lemma 4.2, the probability that the true MIPS results fall outside TAIL is at least $F(\gamma, n_1, n_2)$, since these points are likely dominated by those in HEAD.

Definition 5.1 (δ -NAP). Given a tolerance error δ , and a dataset $\mathcal{D} = \{x_1, x_2, \dots, x_n\}$ sorted by non-increasing norm, the δ -NAP

seeks the maximal value of n_2 such that there exists a subset of n_1 points satisfying $F(\gamma, n_1, n_2) > 1 - \delta$, where $\gamma = \frac{\|x_{n_1}\|}{\|x_{n-n_2+1}\|}$.

However, computing the optimal n_2 for δ -NAP is computationally intensive, as the search space for (n_1, n_2) consists of $O(n^2)$ combinations, and evaluating $F(\gamma, n_1, n_2)$ for each is costly. To overcome this, we next describe practical algorithms that efficiently and effectively perform δ -NAP on real-world datasets.

5.2 Practical NAP Implementation

We consider two MAP algorithms to determine the boundaries of HEAD and TAIL based on the estimated success probability $F(\gamma, n_1, n_2)$.

Greedy δ -NAP. Let $\psi(n_2)$ be the minimal n_1 such that $F(\gamma, n_1, n_2) > 1 - \delta$. Since $F(\gamma, n_1, n_2)$ increases with n_1 and decreases with n_2 for a fixed γ , it is reasonable to assume that $\psi(n_2)$ increases with n_2 , given that $\gamma = \frac{\|x_{n_1}\|}{\|x_{n-n_2+1}\|}$ changes gradually. We can then adopt a greedy strategy to find the appropriate $\psi(n_2)$ for each n_1 . Algorithm 1 outlines the steps of the greedy δ -NAP algorithm. It begins by initializing values for n_1 and n_2 (lines 1). Specifically, n_1 is set to 2048, and n_2 is initialized to 100. We then check whether $F(\gamma, n_1, n_2) > 1 - \delta$ holds in the initial case (line 4). If not, it indicates that all points in $\mathcal{D}$ have nearly identical norms, rendering the NAP unnecessary. In this case, we set $n_1 = n_2 = 0$. Otherwise, we attempt to increase n_2 until $F(\gamma, n_1, n_2) > 1 - \delta$ no longer holds (lines 6 to 9), thereby finding the maximum n_2 that satisfies the tolerance condition. Next, n_1 is incremented (line 6). If $F(\gamma, n_1, n_2)$ decreases with increasing n_1 , the algorithm terminates, as this suggests that the norm reduction from x_{n_1-1} to x_{n_1} (line 11) is too large. Otherwise, we decrease n_1 until $F(\gamma, n_1, n_2) > 1 - \delta$ is satisfied (lines 10 to 15). This process continues until every point is classified as either a HEAD or TAIL point.

Algorithm 1: δ -NAP in $\mathcal{D}$

Input: The sorted dataset $\mathcal{D} = \{x_i\}_{i=1}^n$, parameters δ

Output: The size of HEAD n_1 and TAIL n_2

```

1  $n_1 \leftarrow 2048, n_2 \leftarrow 100;$ 
2  $old_F \leftarrow 0;$ 
3  $\gamma \leftarrow \frac{\|x_{n_1}\|}{\|x_{n-n_2+1}\|};$ 
4 if  $F(\gamma, n_1, n_2) < 1 - \delta$  then
5   return 0,0;
6 while  $n_1 + n_2 < n$  do
7   while  $F(\gamma, n_1, n_2) > 1 - \delta$  do
8      $n_2 \leftarrow n_2 + 1;$ 
9      $\gamma \leftarrow \frac{\|x_{n_1}\|}{\|x_{n-n_2+1}\|};$ 
10  while  $F(\gamma, n_1, n_2) < 1 - \delta$  do
11    if  $F(\gamma, n_1, n_2) < old_F$  then
12      Break;
13     $old_F \leftarrow F(\gamma, n_1, n_2);$ 
14     $n_1 \leftarrow n_1 + 1;$ 
15     $\gamma \leftarrow \frac{\|x_{n_1}\|}{\|x_{n-n_2+1}\|};$ 
16 return  $n_1, n_2 - 1;$ 

```

Heuristic Two-Step δ -NAP. While the greedy δ -NAP algorithm efficiently identifies reasonable values for n_1 and n_2 , it approximates

the norm of points in HEAD as $\|x_{n_1}\|$ and the norm of points in TAIL as $\|x_{n-n_2-1}\|$ when computing the domination probability p . This approach allows for a quick estimation of p , but it tends to significantly underestimate it, especially when there is substantial variation in the norms of the points in HEAD.

To improve this estimation, we consider the more accurate domination probability p based on the Eq. 2, given by:

$$p = \int_{-\infty}^{+\infty} F_{I_Y}^*(t) \cdot F_{I_X}^*(t) dt. \quad (4)$$

Here, $F_{I_X}^*(t)$ represents the probability that $\max_{x \in \text{HEAD}} q^\top x \leq t$, and $F_{I_Y}^*(t)$ represents the probability that $\max_{x \in \text{TAIL}} q^\top x \leq t$, which can be computed as

$$F_{I_X}^*(t) = \prod_{i=1}^{n_1} \Phi\left(\frac{t}{\|x_i\|}\right), F_{I_Y}^*(t) = \prod_{i=n-n_2}^n \Phi\left(\frac{t}{\|x_i\|}\right) \quad (5)$$

The value of p increases monotonically with n_1 and decreases with n_2 , regardless of the norm distribution of $\mathcal{D}$. Thus, the optimal n_2 is found when $p > 1 - \delta$, but $p < 1 - \delta$. Algorithm 2 outlines the steps to determine this optimal n_2 . We adopt a similar iterative approach as in Algorithm 1, but the computation of p is significantly more complex than that of $F(\gamma, n_1, n_2)$. To reduce computational cost, we recursively update $F_{I_Y}^*(t)$, $F_{I_X}^*(t)$, and precompute the derivative of $F_{I_X}^*(t)$, expressed as $F_{I_X}'^*(t) = F_{I_X}^*(t) \cdot \varphi(t)$, where

$$\varphi(t) = \sum_{i=1}^{n_1} \frac{f\left(\frac{t}{\|x_i\|}\right)}{\|x_i\| \cdot \Phi\left(\frac{t}{\|x_i\|}\right)} \quad (6)$$

After determining n_2 , we select the optimal n_1 using the elbow method [29], based on the norm ratio $\gamma = \frac{\|x_{n_1}\|}{\|x_{n-n_2+1}\|}$. The elbow method is employed because it offers a clear indication of where norm domination begins to diminish, helping us choose a balanced value for n_1 that maximizes efficiency without selecting an excessive number of points as HEAD points. This technique enables us to identify the most appropriate n_1 by examining the point where the norm ratio exhibits a sharp change, signaling the optimal trade-off between query accuracy and the high cost of intensive queries in the HEAD region.

6 NAP-based AMIPS Query

After partitioning the dataset $\mathcal{D}$ using NAP, we construct separate indexes for HEAD and BODY. The suitable indexes for HEAD and BODY differ due to their distinct norm distributions and point densities. The BODY region consists of a large number of points with similar norms after the NAP partition, making it ideal to utilize APG as the index. This facilitates the AMIPS query and mitigates norm domination. In contrast, the HEAD region contains fewer points with varied norms. For this reason, it is recommended to use Linscan [1, 35] or LSH-based indexes [17, 42] rather than APGs, for the following reasons: Firstly, the dispersed norm distribution in HEAD makes it extremely difficult to avoid the issue of norm domination. Secondly, the small size of HEAD makes it challenging to find a sufficient number of effective MIPS when compared to the entire dataset. On the other hand, due to its data-independent structure and low construction cost, LSH is sufficient to efficiently

Algorithm 2: Heuristic Two-Step δ -NAP in $\mathcal{D}$

Input: The sorted dataset $\mathcal{D} = \{x_i\}_{i=1}^n$, parameters δ

Output: The size of HEAD n_1 and TAIL n_2

```

1  $n_1 \leftarrow 1$ ;
2  $n_2 \leftarrow 1$ ;
3  $F_{I_X}^*(t) \leftarrow \prod_{i=1}^{n_1} \Phi\left(\frac{t}{\|x_i\|}\right)$ ;
4  $F_{I_Y}^*(t) \leftarrow \prod_{i=n-n_2}^n \Phi\left(\frac{t}{\|x_i\|}\right)$ ;
5  $\varphi(t) \leftarrow \sum_{i=1}^{n_1} \frac{f\left(\frac{t}{\|x_i\|}\right)}{\|x_i\| \cdot \Phi\left(\frac{t}{\|x_i\|}\right)}$ ;
6 while  $n_1 + n_2 < n$  do
7   Compute  $p \leftarrow \int_{-\infty}^{+\infty} F_{I_X}^*(t) \cdot F_{I_Y}^*(t) \cdot \varphi(t) dt$ ;
8   while  $p > 1 - \delta$  do
9      $n_2 \leftarrow n_2 + 1$ ;
10    Update  $F_{I_Y}^*(t) \leftarrow F_{I_Y}^*(t) \cdot \Phi\left(\frac{t}{\|x_{n-n_2-1}\|}\right)$ ;
11    Compute  $p \leftarrow \int_{-\infty}^{+\infty} F_{I_X}^*(t) \cdot F_{I_Y}^*(t) \cdot \varphi(t) dt$ ;
12  while  $p < 1 - \delta$  do
13     $n_1 \leftarrow n_1 + 1$ ;
14    Update  $F_{I_X}^*(t) \leftarrow F_{I_X}^*(t) \cdot \Phi\left(\frac{t}{\|x_{n_1}\|}\right)$ ;
15    Update  $\varphi(t) \leftarrow \varphi(t) + \frac{f\left(\frac{t}{\|x_{n_1}\|}\right)}{\|x_{n_1}\| \cdot \Phi\left(\frac{t}{\|x_{n_1}\|}\right)}$ ;
16    Compute  $p \leftarrow \int_{-\infty}^{+\infty} F_{I_X}^*(t) \cdot F_{I_Y}^*(t) \cdot \varphi(t) dt$ ;
17  Choose  $n_1$  based on  $\frac{\|x_{n_1}\|}{\|x_{n-n_2+1}\|}$  via the elbow method;
18 return  $n_1, n_2$ ;

```

support AMIPS queries in HEAD. Therefore, we propose a practical NAP-based hybrid index, HT-NSW, which integrates LSH [30] and HNSW [28] structures to efficiently support the AMIPS query on top of NAP. Figure 3 illustrates the overall framework of HT-NSW, which comprises three major stages: *Norm-Adaptive Partition*, *Index Construction*, and *Query Processing*. We now describe the construction of HT-NSW and its efficient AMIPS query processing.

6.1 A NAP-based Hybrid Index: HT-NSW

Building Hash Tables for HEAD. LSH is one of the most popular tools for solving the ANNS problem due to its simple structure. Since inner product is not a valid metric, no standard LSH function can be directly applied to AMIPS [17, 30]. Hence, we consider an LSH function based on cosine similarity, *i.e.*, angular distance, to map the points in HEAD, which is defined as follows [30]:

$$h(x) = \text{sign}(a^\top x), \quad (7)$$

where a is a d -dimensional random vector with each entry independently chosen from the standard normal distribution $N(0, 1)$.

LEMMA 6.1. *The collision probability of $h(x)$ and $h(q)$ under the LSH function in Eq. 7 is [42]:*

$$\Pr[h(x) = h(q)] = 1 - \frac{\langle q, x \rangle}{\pi} \quad (8)$$

Lemma 6.1 shows that points with high cosine similarity are more likely to have the same hash value, which helps in identifying the required neighbors from the hash buckets containing q .

An LSH function $h(x)$ alone is insufficient for finding nearest neighbors under cosine similarity effectively, as $\Pr[h(x) = h(q)]$

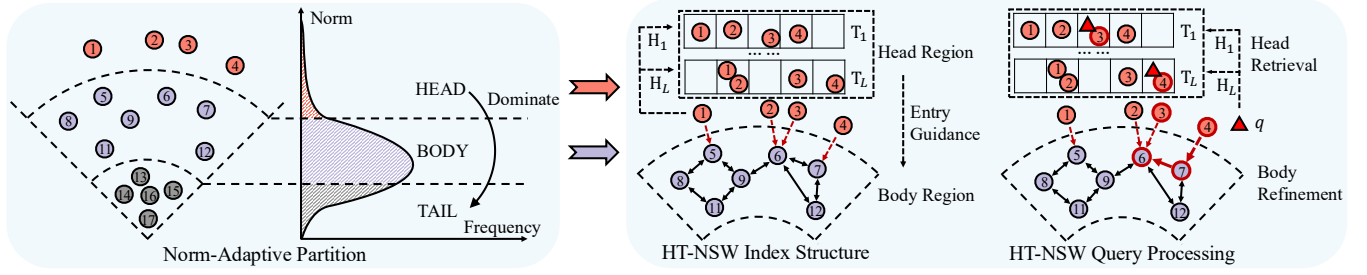


Figure 3: Construction and AMIPS query of HT-NSW.

Algorithm 3: k -AMIPS Query

Input: The sorted dataset $\mathcal{D} = \{x_i\}_{i=1}^n$, the query point q , parameters k , indexes $\mathcal{I}_H$ for HEAD and $\mathcal{I}_B$ for BODY

Output: k -AMIPS of q

- 1 $R \leftarrow k$ -AMIPS neighbors of q found in $\mathcal{I}_H$;
- 2 $R_c \leftarrow k$ -ANNS angular neighbors of q found in $\mathcal{I}_H$;
- 3 **if** $\|q\| \cdot \|x_{n+1}\| < \min_{x \in R} q^\top x$ **then**
- 4 **return** R ;
- 5 $EPs \leftarrow R_c$'s MIPS neighbors in BODY ;
- 6 Search in $\mathcal{I}_H$ with EPs as the entry points;
- 7 $R_B \leftarrow k$ -AMIPS neighbors of q found in $\mathcal{I}_H$;
- 8 Update R with R_B ;
- 9 **return** R ;

can still be very high even when $\langle q, x \rangle$ is large, e.g., $\pi/2$. Therefore, a common strategy in LSH-based methods is to construct several multi-dimensional hash tables [16, 36]. In our HT-NSW method, $\mathcal{I}_H$ consists of L K -dimensional hash tables. We randomly choose K independent hash functions as defined in Eq. 7 for each hash table [30, 36]. Let $g_i(x) = (h_{i,1}(x), \dots, h_{i,K}(x))$ be the hash keys of point x in hash table T_i . We store the pairs $\{g_i(x), x\}$ in T_i .

Building HNSW for BODY. Given that HNSW [27, 38] achieves superior query performance on ANNS, we build the HNSW index for BODY. However, several studies [38, 41] show that the multi-layer structure of HNSW contributes little to query performance, as it typically generates only one point from the upper layer as the entry point for the search in the base layer. To refine the entry point quality and reduce the index size of $\mathcal{I}_B$, we adopt a single-layer HNSW structure for BODY. We then select the entry points based on the k -AMIPS results found in HEAD. We construct the required single-layer HNSW for BODY as outlined in Algorithm 1 of [27]. Specifically, we first create an empty APG. Points are then inserted into the APG one by one. For each insertion, we find the at most M MIPS neighbors of point x and mutually connect x to its neighbor y . When the out-degree of point y exceeds $2M$ (indicating that y has been selected as a neighbor for other points), we select M new neighbors for y from its old neighbor set.

6.2 NAP-based AMIPS Query Algorithm

After indexing HEAD with $\mathcal{I}_H$ and BODY with $\mathcal{I}_B$, we perform the AMIPS query for q as shown in Algorithm 3. We first identify the k -AMIPS neighbors of q in $\mathcal{I}_H$ (line 1). Next, we examine whether better k -AMIPS neighbors exist in BODY (line 3). Since x_{n+1} is the

Algorithm 4: k -AMIPS Query in $\mathcal{I}_H$

Input: The query q , parameters k , indexes $\mathcal{I}_H$ for HEAD

Output: Two sets of points

- 1 $R \leftarrow \emptyset$; **▷ Store the found best k -AMIPS of q**
- 2 $R_c \leftarrow \emptyset$; **▷ Store the found best angular k -ANNS of q**
- 3 **for** $l \leftarrow 0 : K$ **do**
- 4 **for each** hash table T_i **do**
- 5 **if** $\|g_i(x), g_i(q)\|_H = l - 1$ **then**
- 6 Update R with x ;
- 7 Update R_c with x ;
- 8 **if** $\|q\| \cdot M_f \cos \theta_l(\delta) \leq \min_{x \in R} q^\top x$ **then**
- 9 **return** R, R_c ;
- 10 **return** R, R_c ;

Algorithm 5: k -AMIPS in $\mathcal{I}_B$

Input: A query point q , the APG $\mathcal{I}_B$, the entry point set EPs , the parameters k and ef

Output: k points in $\mathcal{I}_B$

- 1 $R \leftarrow \emptyset$;
- 2 **while** $|EPs| > 0$ **do**
- 3 $ep \leftarrow$ extract q 's best MIPS results from EPs ;
- 4 **if** $|R| \geq ef$ and $q^\top ep < q^\top x_f$ **then**
- 5 **break**;
- 6 **for** $x \in ep$'s neighbor set **do**
- 7 **if** x is unvisited **then**
- 8 $EPs \leftarrow EPs \cup \{x\}$, $R \leftarrow R \cup \{x\}$;
- 9 **while** $|R| > ef$ **do**
- 10 Remove q 's worst MIPS results in R ;
- 11 $x_f \leftarrow \arg \min_{x \in R} q^\top x$;
- 12 $R \leftarrow$ The best k MIPS results to q in R ;
- 13 **return** R ;

point with the largest norm in BODY, the product $\|q\| \cdot \|x_{n+1}\|$ serves as the upper bound for the inner product between q and any point in BODY. Thus, if $\|q\| \cdot \|x_{n+1}\| < \min_{x \in R} q^\top x$, we can safely terminate the query without further searching in $\mathcal{I}_B$. Otherwise, we use the results obtained from $\mathcal{I}_H$ to select appropriate entry points for the search in $\mathcal{I}_B$ and then generate the final k -AMIPS results for q from both $\mathcal{I}_H$ and $\mathcal{I}_B$ (line 7). We now describe the query algorithms in $\mathcal{I}_H$ and $\mathcal{I}_B$, including the entry point selection for the search in $\mathcal{I}_B$.

AMIPS & ANNS Query in $\mathcal{I}_H$. Unlike traditional LSH-based methods for ANNS, $\mathcal{I}_H$ is built over angular distance rather than inner product. The conventional LSH-based query strategy, which probes candidates only from q 's hash buckets, typically achieves limited query accuracy. To ensure an accurate and exhaustive query in $\mathcal{I}_H$, we adopt the more precise multi-probing strategy [26] for the search in $\mathcal{I}_H$, and we terminate the query adaptively by estimating the angle between q and x based on their hash values. Algorithm 4 describes our search strategy in $\mathcal{I}_H$. Specifically, we conduct at most $K + 1$ rounds of search in $\mathcal{I}_H$ (lines 3-9). Since $g_i(x)$ is a K -dimensional binary vector, the Hamming distance $\|g_i(x), g_i(q)\|_H$ lies within the interval $[0, K]$, with a larger $\|g_i(x), g_i(q)\|_H$ corresponding to a larger angle $\langle q, x \rangle$, i.e., lower cosine similarity. Thus, at the l -th round of search, we probe the buckets $h_l(x)$ in T_i where $\|g_i(x), g_i(q)\|_H = l - 1$ (line 5). For each found point x when probing a bucket, we compute the angular distance $\langle q, x \rangle$ and inner product $q^\top x$, updating the results into R_c and R , respectively. R_c is used for generating high-quality entry points for the search in $\mathcal{I}_B$. Computing $\langle q, x \rangle$ incurs no additional cost since it can be derived from $q^\top x$. Finally, we consider adaptively terminating the query based on the properties of the LSH functions.

LEMMA 6.2. *Given a tolerant error δ , let $\theta_l(\delta)$ be the solution to the following equation on θ ,*

$$\left[1 - \sum_{j=0}^l \binom{K}{j} \cdot \left(1 - \frac{\theta}{\pi}\right)^{K-j} \cdot \left(\frac{\theta}{\pi}\right)^j\right]^L = \delta. \quad (9)$$

If the point x satisfies $\langle q, x \rangle \leq \theta_l(\delta)$, the probability that it is found during the first $l + 1$ rounds of search is at least $1 - \delta$.

The proof of Lemma 6.2 can be found in Section 7.1. Based on this lemma, we terminate the search in $\mathcal{I}_H$ after $l + 1$ rounds of search if $\|q\| \cdot M_j \cos \theta_l(\delta)$ is smaller than the best k results found so far (line 8), where M_j represents the largest norm of the unseen points in HEAD . When this condition is met, the probability of finding a better MIPS result for q from other hash buckets becomes extremely small.

AMIPS Query in $\mathcal{I}_B$. The k -AMIPS query for q in $\mathcal{I}_B$ follows the greedy search approach shown in Algorithm 5. In each search epoch, the point ep that has the maximum inner product with q in EPs is selected as the next entry point (Line 3). For each neighbor x of ep in the APG, we compute its inner product with q and update the results into the sets EP and R . The result set R stores the best ef points found so far. The search procedure repeats until no point in EPs has a larger inner product with q than any of the best ef points in R (Line 4). At that point, the search terminates, and the top k points in R are returned as the AMIPS results for q . The parameter ef is set to balance query efficiency and accuracy. A higher ef makes termination more difficult (Line 4), allowing us to explore more points to refine the query results, but it increases query cost.

Head-Guided Entry Point Generation. To enhance the query efficiency of the search in $\mathcal{I}_B$, we generate high-quality entry points based on the k -AMIPS results found in HEAD . First, for each point x in HEAD , we retrieve its AMIPS results in BODY ahead and store them in the index during the indexing phase. Then, during the query phase, after the search in $\mathcal{I}_H$, we have found q 's angular neighbors in HEAD , i.e., R_c (Algorithm 4, line 7). For each point x

in R_c , we add its MIPS neighbors y in $\mathcal{I}_B$ to the entry point set EPs . Although the inner product is not a metric, it is expected that point y will have a large inner product with q , because the MIPS neighbor of an angular neighbor is likely to also be a MIPS neighbor [25]. Finally, we illustrate the overall AMIPS query process in HT-NSW through the example in Figure 3.

Example 6.3. Given a query point q (the red triangle), the right subgraph in Figure 3 illustrates the complete k -AMIPS query in HT-NSW with $k = 1$. The query first probes the HEAD index $\mathcal{I}_H$ round by round. The retrieved candidates from $\mathcal{I}_H$ are marked with a red face in the figure, i.e., point x_3 and x_4 where x_4 is the MIPS result of q in HEAD . Assume the inner product between x_4 and q is less than $\|q\| \cdot M$, where M is the largest norm of points in BODY , indicating that the points in BODY are not dominated by x_4 and the query needs to continue in $\mathcal{I}_B$. Then, we generate the entry point in BODY based on the nearest neighbors of q in HEAD under cosine similarity, i.e., x_4 . Finally, we conduct the search in the APG to find the AMIPS of q in BODY , with x_7 as the entry point, since it is the MIPS neighbor of x_4 in BODY . The search terminates after accessing x_7 , as its inner product with q is larger than its neighbor in the APG, i.e., x_6 .

7 Theoretical Analysis

We begin by presenting the proofs of the lemmas introduced in Sections 4–6. Specifically, Lemma 4.1 quantifies the norm bias phenomenon between two points; Lemma 4.2 further extends this analysis by demonstrating the norm domination effect among multiple points, based on Lemma 4.1; and Lemma 6.2 establishes a theoretical bound for the search process within $\mathcal{I}_H$. Subsequently, we analyze both the space and time complexity of the HT-HNSW framework.

7.1 Proofs of Lemmas

PROOF OF LEMMA 4.1. We aim to demonstrate that

$$\Pr[q^\top x_1 > q^\top x_2 \mid q^\top x_1 > 0, q^\top x_2 > 0] = \frac{2}{\pi} \cdot \arctan \frac{r_1}{r_2}. \quad (10)$$

Based on Lemma 3.3, we have $I_1 = q^\top x_1 \sim N(0, r_1^2)$ and $I_2 = q^\top x_2 \sim N(0, r_2^2)$. Since x_1 and x_2 are orthogonal, the covariance between I_1 and I_2 is 0, indicating that I_1 and I_2 are uncorrelated. By Lemma 3.4, I_1 and I_2 are independent. So,

$$\Pr[q^\top x_1 > 0, q^\top x_2 > 0] = \Pr[q^\top x_1 > 0] \cdot \Pr[q^\top x_2 > 0] = \frac{1}{4}.$$

Let $f(t) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{t^2}{2\sigma^2}}$ denote the probability density function (pdf) of a standard normal distribution. Then,

$$\Pr[q^\top x_1 > q^\top x_2 > 0] = \int_0^{+\infty} \int_0^{t_1} \frac{1}{2\pi r_1 r_2} e^{-\frac{1}{2}\left(\frac{t_1^2}{r_1^2} + \frac{t_2^2}{r_2^2}\right)} dt_1 dt_2.$$

Perform the substitution $t_1 = r_1 r \cos \theta$, $t_2 = r_2 r \sin \theta$, where $r \in (0, +\infty)$ and $\theta \in (0, u)$ with $u = \arctan(r_1/r_2)$. Then,

$$\begin{aligned} \Pr[q^\top x_1 > q^\top x_2 > 0] &= \frac{1}{2\pi} \int_0^{+\infty} e^{-\frac{r^2}{2}} r dr \cdot \int_0^u d\theta \\ &= \frac{1}{2\pi} u = \frac{1}{2\pi} \arctan \frac{r_1}{r_2}. \end{aligned}$$

$$\begin{aligned} & \text{and } \Pr[q^\top x_1 > q^\top x_2 \mid q^\top x_1 > 0, q^\top x_2 > 0] \\ &= \frac{\Pr[q^\top x_1 > q^\top x_2 > 0]}{\Pr[q^\top x_1 > 0, q^\top x_2 > 0]} = \frac{2}{\pi} \cdot \arctan \frac{r_1}{r_2}. \end{aligned}$$

□

PROOF OF LEMMA 4.2. When $t > 0$, $\Phi(t)$ increases monotonically with t , then $\Phi(\frac{t}{\|x_i\|}) < \Phi(\frac{t}{r_1})$ and $\Phi(\frac{t}{\|y_i\|}) > \Phi(\frac{t}{r_2})$. So, $F_{I_X}^*(t) < \Phi^{n_1}(\frac{t}{r_1})$ and $F_{I_Y}^*(t) > \Phi^{n_2}(\frac{t}{r_2})$. Therefore, we have

$$\begin{aligned} \Psi(X, Y) &= \int_{-\infty}^{+\infty} \int_{t_1}^{+\infty} F_{I_X}^*(t_2) dt_2 \cdot F_{I_Y}^*(t_1) dt_1 \\ &> \int_0^{+\infty} (1 - \Phi^{n_1}(\frac{t_1}{r_1})) \cdot F_{I_Y}^*(t_1) dt_1 \\ &> (1 - \Phi^{n_1}(\frac{t_1}{r_1})) \cdot \Phi^{n_2}(\frac{t_1}{r_2})|_{t_1=0}^{+\infty} + \int_0^{+\infty} \Phi^{n_2}(\frac{t_1}{r_2}) d\Phi^{n_1}(\frac{t}{r_1}) \\ &= \int_0^{+\infty} [1 - \Phi^{n_1}(t/\gamma)] d\Phi^{n_2}(t) \quad \text{Let } t = t_1/r_2 \end{aligned}$$

□

PROOF OF LEMMA 6.2. Based on Lemma 6.1, if $\theta_0 = \langle q, x \rangle \leq \theta_l(\delta)$, then for any a function h defined as Eq. 7, we have $\Pr[h(q) = h(x)] \geq 1 - \frac{\theta_l(\delta)}{\pi}$. Then, we consider the following events.

- E_i : x is unvisited in the table T_i at the first $l+1$ rounds of probing.
- E_{all} : x is unvisited in all L tables at the first $l+1$ rounds of probing.

Then, we have

$$\Pr[E_i] = \Pr[\|g_i(x), g_i(q)\|_H > l] = 1 - \Pr[\|g_i(x), g_i(q)\|_H \leq l]$$

As $\Pr[\|g_i(x), g_i(q)\|_H = j] = \binom{K}{j} \left(\frac{\theta_0}{\pi}\right)^j \cdot \left(\frac{\theta_0}{\pi}\right)^{K-j}$, so

$$\Pr[E_i] = 1 - \sum_{j=0}^l \binom{K}{j} \cdot \left(1 - \frac{\theta_0}{\pi}\right)^j \cdot \left(\frac{\theta_0}{\pi}\right)^{K-j}$$

and $\Pr[E_{\text{all}}] = \Pr[E_1 \wedge \dots \wedge E_L]$

$$= \prod_{i=1}^L \Pr[E_i] = \left(1 - \sum_{j=0}^l \binom{K}{j} \cdot \left(1 - \frac{\theta_0}{\pi}\right)^j \cdot \left(\frac{\theta_0}{\pi}\right)^{K-j}\right)^L.$$

Since $p(\theta) = [1 - \sum_{j=0}^l \binom{K}{j} \cdot \left(1 - \frac{\theta}{\pi}\right)^j \cdot \left(\frac{\theta}{\pi}\right)^{K-j}]^L$ increases with θ and $p(\theta_l(\delta)) = \delta$, so $g(\theta_0) \leq \delta$. □

7.2 Complexity Analysis

We analyze the construction cost, space consumption, and query cost of HT-NSW in this subsection. The construction cost of HT-NSW is determined by the cost of NAP and the costs associated with constructing I_H and I_B . The NAP process can be completed in at most n iterations, with each iteration having a constant cost of $O(1)$, resulting in an overall NAP cost of $O(n)$. The structure of I_H consists of L K -dimensional hash tables. Typically, we set $L = O(1)$ and $K = O(\log n_1)$ to ensure that, on average, each bucket contains exactly one point. Consequently, the time required to compute the hash values is $O(n_1 L K) = O(n_1 \log n_1)$, and the time to construct the hash tables is $O(n)$. The complexity of I_B is governed by HNSW, whose empirical query cost is $O(\log N)$, where N represents the number of points in HNSW [27]. The construction of HNSW in BODY involves $n - n_1 - n_2$ insertions, with the cost of these insertions

Table 1: Summary of Datasets

Datasets	Card.	Dim.	Attribute Type
Audio	54,187	192	Audio embedding
MNIST	60,000	784	Raw image
YahooMusic	625,000	300	Rating decomposition
Gist1M	1,000,000	960	GIST descriptor
Tiny5M	5,000,000	384	Image embedding

dominated by the query cost in the APG. Let $\beta = \frac{n - n_1 - n_2}{n}$, the construction cost of I_B is then $O(\beta n \log(\beta n)) = O(\beta n \log(n))$. Thus, the total construction cost of HT-NSW is $O(n + n_1 \log n_1 + \beta n \log(n))$. Since $n_1 \ll n$, the construction cost of HT-NSW can be simplified to $O(\beta n \log(n))$. Similarly, the space consumption of HT-NSW is determined by the sizes of I_H and I_B , both of which are linear in the number of indexed points. Therefore, the space consumption of HT-NSW is $O(n)$. The worst-case query cost of HT-NSW occurs when all points in I_H are accessed, and the search continues within I_H . In this scenario, the query cost is $O(\log n + n_1) = O(\log n)$, provided that n_1 remains sufficiently small.

In summary, HT-NSW can be constructed within $O(\beta n \log(n))$ time and requires $O(n)$ space, where β is the ratio of the number of points in BODY. HT-NSW answers AMIPS queries in $O(\log n)$ time.

8 Experiments

In this section, we present a series of extensive experiments on real-world datasets to provide a comprehensive analysis of the NAP framework and HT-NSW. The HT-NSW implementation¹ and its competitors are developed in C++. All experiments are conducted on an Ubuntu 22.04.5 LTS system equipped with 4 Intel(R) Xeon(R) Gold 6218 CPUs and 1.5 TB of RAM.

8.1 Experimental Settings

Datasets. We employ five real-world datasets that are widely used in Approximate Nearest Neighbor Search (ANNS) and Approximate Maximum Inner Product Search (AMIPS) [25, 28, 42], the details of which are summarized in Table 1 in ascending order of their sizes. We present their norm distribution and MIPS distribution in Figure 1. For each dataset, we randomly select 10K query points from its query set to evaluate the AMIPS query performance.

Competitors. We compare HT-NSW with several state-of-the-art graph-based methods for AMIPS.

- **HNSW** [27] and **ip-NSW**² [28] share the same construction strategy, with the key distinction being that HNSW uses L_2 distance as the similarity function, whereas ip-NSW uses the *inner product*. The underlying structure is identical to that in HT-NSW.
- **NAPG**³ [33] enhances ip-NSW by introducing an adaptive edge selection strategy.
- **PSP**⁴ [7] scales query q to $q' = \mu \cdot q$ and demonstrates that the NNS of q' in dataset $\mathcal{D}$ corresponds to the MIPS of q in $\mathcal{D}$. They answer the AMIPS query for q by constructing the APG in Euclidean space and performing a search on q' .

Parameter Settings. In our default configuration, we set $k = 10$ for all algorithms and adopt fixed parameters for each method. For

¹<https://github.com/SIGMOD2026-NAP/SIGMOD2026-NAP>

²<https://github.com/stanis-morozov/ip-nsw>

³https://github.com/ThrunGroup/BanditMIPS/tree/camera_ready/algorithms/napg

⁴<https://github.com/ZJU-DAILY/PSP>

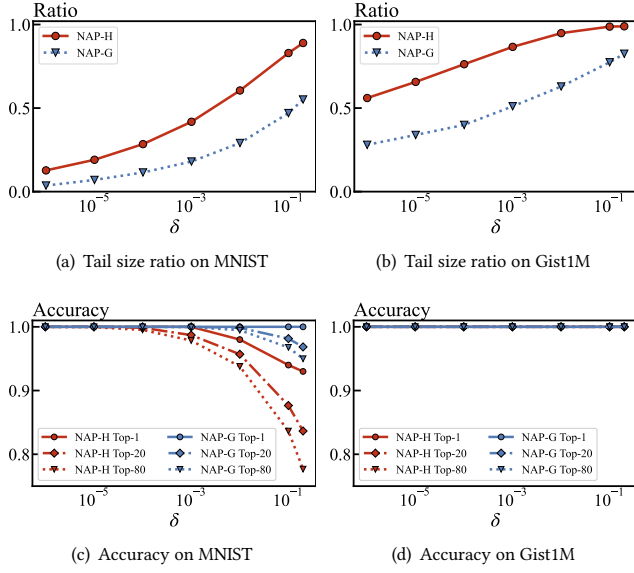


Figure 4: Comparison between greedy and heuristic NAP.

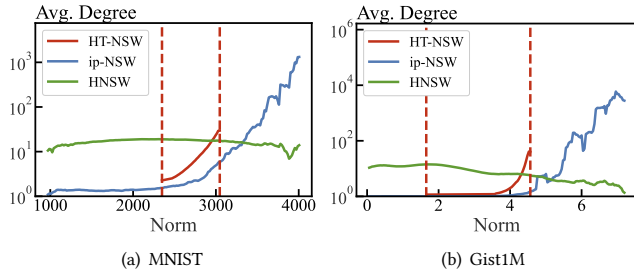


Figure 5: Degree-Norm Distribution.

HT-NSW, we use the heuristic NAP strategy with $\delta = 0.001$ to partition the dataset. Additionally, we set $L = 5$ and $K = \log n_1$ for $\mathcal{I}_H$, and $M = 24$ and $efc = 80$ for $\mathcal{I}_B$. For both HNSW and NAPG, we use the same settings, i.e., $M = 24$ and $efc = 80$. For PSP, we configure $R = 40$, $\alpha = 60$, and $S = 50$ for all datasets.

Evaluation Metrics. The query performance of each algorithm is evaluated using *QPS* (queries per second) and *recall@k* with a single thread. The *recall@k* is defined as follows [7], $Recall = \frac{|R \cap R^*|}{k}$ where R denotes the top- k AMIPS results returned by the algorithm, and R^* represents the exact top- k MIPS results for query q . Furthermore, we assess the indexing performance of each algorithm using *indexing time* and *index size* with a total of 160 threads.

8.2 Self Evaluation of NAP

In this subsection, we demonstrate the effectiveness of NAP. Firstly, by varying the tolerance error δ , we compare greedy NAP and the heuristic two-step NAP in terms of their tail size and the accuracy loss incurred during partitioning. Secondly, we analyze the impact of NAP on the APG structure by examining the edge distribution of HT-NSW, HNSW (for ANNS), and ip-NSW (for AMIPS).

8.2.1 Evaluation of NAP Strategies. Figure 4 presents a comparison between the greedy and heuristic two-step variants of NAP

while varying the tolerance error δ from 10^{-6} to 0.2. The upper row illustrates the resulting ratio of TAIL, n_2/n . The bottom row reports the accuracy of NAP, measured by the proportion of Top- k MIPS results retained within the HEAD and BODY after NAP. A higher proportion indicates that more MIPS results are preserved in these regions, suggesting that the exclusion of tail points incurs less accuracy loss. The experimental results on both datasets reveal consistent trends regarding the influence of the tolerance parameter δ on the performance of the two NAP strategies.

(1) **Increasing δ enlarges the tail region.** For instance, on the Audio dataset, the ratio of tail in NAP-H grows from 13% to 90% as δ rises from 10^{-6} to 0.2. A similar trend is observed on Gist1M (28% $\rightarrow$ 80% for NAP-G and 55% $\rightarrow$ 99% for NAP-H). This monotonic increase demonstrates that both NAP strategies are capable of expanding the partitioning boundary with higher tolerance error, thereby striking a balance between query accuracy and index size.

(2) **High accuracy retention may occur under wide tolerance.** Empirically, both NAP strategies maintain nearly perfect accuracy when $\delta \leq 10^{-3}$, with only slight degradation in accuracy as δ increases on the Audio dataset. However, on Gist1M, no accuracy loss is observed at any value of δ . This observation aligns with the norm domination phenomenon discussed in Section 4, where large-norm vectors dominate MIPS results. By adjusting the boundaries between the HEAD, BODY, and TAIL according to the tolerance error δ , NAP effectively identifies the range of dominant points, providing a self-adaptive framework that minimizes accuracy loss during pruning the unqualified TAIL points.

(3) **Comparison of NAP-H and NAP-G.** The heuristic two-step NAP (NAP-H) consistently identifies more TAIL points than greedy NAP (NAP-G), albeit at the cost of a higher accuracy loss. The performance difference between the two strategies stems from their underlying estimation mechanisms. The heuristic two-step NAP explicitly re-estimates the domination probability p based on the precise norm distribution, leading to more aggressive pruning when the tolerance δ is small. This enables it to effectively identify and discard redundant tail vectors, achieving stronger compression with an acceptable accuracy loss. Moreover, the accuracy loss is minimal when $\delta \leq 0.001$. Therefore, we recommend using the heuristic two-step NAP with $\delta = 0.001$ as the default NAP strategy.

8.2.2 Effect of NAP on APG Structure. To further investigate how NAP and norm domination affect the underlying APG structure, we compare the in-edge distribution across norm regions in HT-NSW, HNSW (under Euclidean space), and ip-NSW (under inner product space). The results are shown in Figure 5.

(1) HNSW, originally designed for indexing points in metric space, maintains a nearly uniform degree distribution across all norm ranges. Most nodes have in-degrees concentrated within a narrow range of 10–80 on both datasets. This stability arises from its metric-space assumption: since Euclidean distance satisfies the triangle inequality, graph construction relies on local proximity rather than vector magnitude. However, such a structure is inherently restricted to metric-based similarity search and cannot be directly applied to MIPS queries. As such, it serves as a useful reference, representing an idealized graph topology with a well-balanced edge distribution. (2) In contrast, ip-NSW adapts HNSW by replacing Euclidean distance with the inner product. The in-degree distribution of ip-NSW exhibits a steep positive correlation

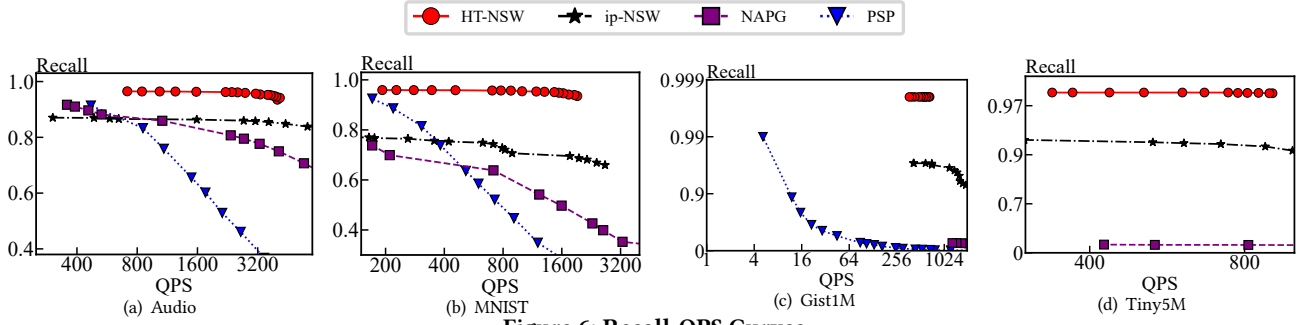


Figure 6: Recall-QPS Curves

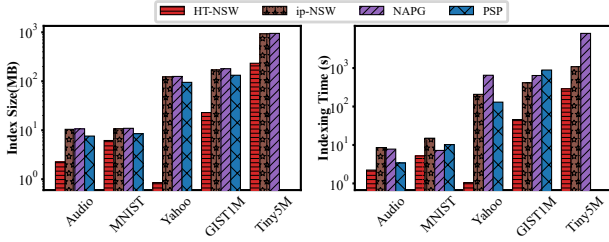


Figure 7: Index Size and Indexing Time

with the norm of points: the average in-degree increases rapidly from near-zero for small-norm nodes to above 10^3 for large-norm nodes. This skewed distribution corroborates the norm domination phenomenon discussed in Section 4.3—large-norm vectors dominate the edges, acting as hubs, while small-norm vectors become nearly isolated. Such an imbalanced topology disrupts global navigability and limits query performance. (3) Our proposed HT-NSW integrates NAP with the proximity graph to mitigate the imbalance caused by norm domination. HT-NSW indexes only the points in the BODY, whose norm range is much smaller than that of the entire dataset, thereby alleviating the norm domination issue. As shown in Figure 5, although the in-degree distribution of HT-NSW is not as uniform as that of HNSW, it is significantly more balanced compared to ip-NSW. This suggests that NAP effectively redistributes connections, preventing hub overload and ensuring efficient navigation.

8.3 Comparison of HT-NSW with Baselines

In this set of experiments, we evaluate both the indexing and query performance of HT-NSW in comparison to the baseline methods. Since PSP fails to build an index on the Tiny5M dataset, its results are not included for this particular dataset.

8.3.1 Comparison of Indexing Performance. The indexing results for all algorithms are presented in Figure 7, comparing them in terms of index size and indexing time. It is evident that HT-NSW achieves significantly smaller index sizes and indexing times compared to the other methods. Notably, on the Yahoo dataset (i.e., YahooMusic), the index size and indexing time of HT-NSW are only about 1% of those of ip-NSW. Given that HT-NSW adopts the same graph structure as ip-NSW, this performance improvement can be attributed to the NAP, in line with the observations in Section 8.2.1. As for the baselines, all of them exhibit similar index sizes on the same dataset. This similarity arises because the size of the APG is largely dependent on the maximum degree, and these methods

share comparable degrees. However, despite the similar index sizes, NAPG exhibits a significantly larger indexing time than ip-NSW on YahooMusic, Gist1M, and Tiny5M, but a smaller indexing time on Audio and MNIST. The performance of NAPG, which enhances ip-NSW through a new edge selection strategy, indicates that its improvements are not universally effective.

8.3.2 Comparison of Query Performance. We assess the query performance of all algorithms using recall-QPS curves, as shown in Figure 6. For a given QPS, an algorithm with a higher recall is considered a better AMIPS method. To obtain varying QPS and recall values for these graph-based methods, we adjust the value of ef in Algorithm 5. From the figures, we observe the following: (1) Among all algorithms, HT-NSW achieves a much higher recall than the others at the same QPS, indicating superior query performance. Furthermore, the recall of HT-NSW approaches 1 across all datasets. This is because HT-NSW identifies most of the AMIPS results within the HEAD, allowing for extremely high recall even when searching within the BODY at minimal cost. (2) The recall of HNSW and NAPG does not reach the levels of HT-NSW, even when searching a larger number of points. This provides strong evidence of norm domination, consistent with the findings in Section 8.2.2. The recall of NAPG is particularly low in Gist1M and Tiny5M, highlighting the limitations of its improvements over HNSW. (3) PSP can mitigate the norm domination issue in Audio and MNIST, but its recall is lower than that of other algorithms at the same QPS. To achieve a higher recall, PSP incurs much larger query cost compared to HT-NSW. This demonstrates that simply increasing the number of query points is not sufficient to effectively address AMIPS, due to the out-of-distribution issue.

9 Conclusion

This paper addresses the critical challenge of norm bias in graph-based Approximate Maximum Inner Product Search (AMIPS). We identify the norm domination phenomenon as a fundamental limitation and propose the Norm-Adaptive Partitioning (NAP) framework to mitigate it. NAP strategically partitions data into HEAD, BODY, and TAIL segments, applying tailored search strategies to each: exhaustive search for HEAD, safe pruning for TAIL, and graph-based search for BODY. This approach led to a hybrid index that reduces the memory overhead of existing graph structures by more than 50% and achieves more than 2x query speedup while maintaining target recall. Our work demonstrates that NAP effectively unlocks the full potential of proximity graphs for efficient and scalable AMIPS.

References

- [1] Firas Abuzaied, Geet Sethi, Peter Bailis, and Matei Zaharia. 2019. To Index or Not to Index: Optimizing Exact Maximum Inner Product Search. In *ICDE*. IEEE, 1250–1261.
- [2] Thomas Dybdahl Ahle, Rasmus Pagh, Ilya P. Razenshteyn, and Francesco Silvestri. 2016. On the Complexity of Inner Product Similarity Join. In *PODS*. 151–164.
- [3] Imad Aouali, Amine Benhaloum, Martin Bompairé, Achraf Ait Sidi Hammou, Sergey Ivanov, Benjamin Heymann, David Rohde, Otmane Sakhi, Flavian Vasile, and Maxime Vono. 2022. Reward Optimizing Recommendation using Deep Learning and Fast Maximum Inner Product Search. In *KDD*. ACM, 4772–4773.
- [4] Yoram Bachrach, Yehuda Finkelstein, Ran Gilad-Bachrach, Liran Katzir, Noam Koenigstein, Nir Nave, and Ulrich Paquet. 2014. Speeding up the Xbox recommender system using a Euclidean transformation for inner-product spaces. In *RecSys*. 257–264.
- [5] Lijie Chen. 2018. On The Hardness of Approximate and Exact (Bichromatic) Maximum Inner Product. *Electronic Colloquium on Computational Complexity (ECCC)* 25 (2018), 26.
- [6] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X. Sean Wang. 2024. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 17, 11 (2024), 2735–2749.
- [7] Tingyang Chen, Cong Fu, Kun Wang, Xiangyu Ke, Yunjun Gao, Wenqiao Zhou, Yabo Ni, and Anxiang Zeng. 2025. Maximum Inner Product is Query-Scaled Nearest Neighbor. *Proc. VLDB Endow.* 18, 6 (2025), 1770–1783.
- [8] Normal distribution. 2025. Available at: https://en.wikipedia.org/wiki/Normal_distribution
- [9] Normal distribution. 2025. Available at: https://en.wikipedia.org/wiki/Sum_of_normally_distributed_random_variables
- [10] Normal distribution. 2025. Available at: https://www.probabilitycourse.com/chapter5/5_3_2_bivariate_normal_dist.php
- [11] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *WWW*. 577–586.
- [12] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. In *KDD*. ACM, 6491–6501.
- [13] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *PVLDB* 12, 5 (2019), 461–474.
- [14] Junhao Gan, Jianlin Feng, Qiong Fang, and Wilfred Ng. 2012. Locality-sensitive hashing scheme based on dynamic collision counting. In *SIGMOD*. 541–552.
- [15] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 3 (2024), 167.
- [16] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *VLDB*. 518–529.
- [17] Qiang Huang, Guihong Ma, Jianlin Feng, Qiong Fang, and Anthony K. H. Tung. 2018. Accurate and Fast Asymmetric Locality-Sensitive Hashing Scheme for Maximum Inner Product Search. In *KDD*. 1561–1570.
- [18] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *STOC*. 604–613.
- [19] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (2011), 117–128.
- [20] Jyun-Yu Jiang, Patrick H. Chen, Cho-Jui Hsieh, and Wei Wang. 2020. Clustering and Constructing User Coresets to Accelerate Large-scale Top-K Recommender Systems. In *WWW*. ACM / IW3C2, 2177–2187.
- [21] Alexis Joly and Olivier Buisson. 2011. Random maximum margin hashing. In *CVPR*. IEEE Computer Society, 873–880.
- [22] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *EMNLP (1)*. Association for Computational Linguistics, 6769–6781.
- [23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.
- [24] Sheng-Chieh Lin, Chankyu Lee, Mohammad Shoeybi, Jimmy Lin, Bryan Catanzaro, and Wei Ping. 2025. Mm-Embed: Universal Multimodal Retrieval with Multimodal LLMs. In *ICLR*. OpenReview.net.
- [25] Jie Liu, Xiao Yan, Xinyan Dai, Zhirong Li, James Cheng, and Ming-Chang Yang. 2020. Understanding and Improving Proximity Graph Based Maximum Inner Product Search. In *AAAI*. 139–146.
- [26] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2007. Multi-Probe LSH: Efficient Indexing for High-Dimensional Similarity Search. In *VLDB*. 950–961.
- [27] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.
- [28] Stanislav Morozov and Artem Babenko. 2018. Non-metric Similarity Graphs for Maximum Inner Product Search. In *NeurIPS*. 4726–4735.
- [29] Erich Schubert. 2023. Stop using the elbow criterion for k-means and how to choose the number of clusters instead. *SIGKDD Explor.* 25, 1 (2023), 36–42.
- [30] Anshumali Shrivastava. 2015. Improved Asymmetric Locality Sensitive Hashing (ALSH) for Maximum Inner Product Search (MIPS). In *UAI*. AUAI Press, 812–821.
- [31] Anshumali Shrivastava and Ping Li. 2014. Asymmetric LSH (ALSH) for Sublinear Time Maximum Inner Product Search (MIPS). In *NIPS*. 2321–2329.
- [32] Suhas Jayaram Subramanya, Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *NeurIPS*. 13748–13758.
- [33] Shulong Tan, Zhaozhuo Xu, Weijie Zhao, Hongliang Fei, Zhixin Zhou, and Ping Li. 2021. Norm Adjusted Proximity Graph for Fast Inner Product Retrieval. In *KDD*. ACM, 1552–1560.
- [34] Shulong Tan, Zhixin Zhou, Zhaozhuo Xu, and Ping Li. 2019. On Efficient Retrieval of Top Similarity Vectors. In *EMNLP/TJCNLP (1)*. Association for Computational Linguistics, 5235–5245.
- [35] Christina Teflioudi and Rainer Gemulla. 2017. Exact and Approximate Maximum Inner Product Search with LEMP. *ACM Trans. Database Syst.* 42, 1 (2017), 5:1–5:49.
- [36] Yao Tian, Xi Zhao, and Xiaofang Zhou. 2022. DB-LSH: Locality-Sensitive Hashing with Query-based Dynamic Bucketing. In *ICDE*. 2250–2262.
- [37] Zhoujin Tian, Chaozhuo Li, Zhiqiang Zuo, Zengxuan Wen, Lichao Sun, Xinyue Hu, Wen Zhang, Haizhen Huang, Senzhang Wang, Weiwei Deng, Xing Xie, and Qi Zhang. 2023. PASS: Personalized Advertiser-aware Sponsored Search. In *KDD*. ACM, 4924–4936.
- [38] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *PVLDB* 14, 11 (2021), 1964–1978.
- [39] Cong Wei, Yang Chen, Haonan Chen, Hexiang Hu, Ge Zhang, Jie Fu, Alan Ritter, and Wenhui Chen. 2024. UniLR: Training and Benchmarking Universal Multimodal Information Retrievers. In *ECCV (87) (Lecture Notes in Computer Science, Vol. 15145)*. Springer, 387–404.
- [40] Xiao Yan, Jinfeng Li, Xinyan Dai, Hongzhi Chen, and James Cheng. 2018. Norm-Ranging LSH for Maximum Inner Product Search. In *NeurIPS*. 2956–2965.
- [41] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *Proc. VLDB Endow.* 16, 8 (2023), 1979–1991.
- [42] Xi Zhao, Bolong Zheng, Xiaomeng Yi, Xiaofan Luan, Charles Xie, Xiaofang Zhou, and Christian S. Jensen. 2023. FARGO: Fast Maximum Inner Product Search via Global Multi-Probing. *Proc. VLDB Endow.* 16, 5 (2023), 1100–1112.
- [43] Zhixin Zhou, Shulong Tan, Zhaozhuo Xu, and Ping Li. 2019. Möbius Transformation for Fast Inner Product Search on Graph. In *NeurIPS*. 8216–8227.